Jade Lambertz

Sprint 3

CS449 – Foundations of Software Engineering

1. **Demonstration Video: Sprint3**

2. **Summary of Source Code**

Source code file name	Production code or test code?	# lines of code
Board.py	Production	138
Game.py	Production	151
GameScreen.py	Production	617
Graphics.py	Production	245
GUI.py	Production	120
Main.py	Production	12
Recording.py	Production	43
SetupScreen.py	Production	334
Manual_board_test.py	Test	25
Test_automated_game.py	Test	62
Test_board.py	Test	96
Test_game.py	Test	129
Test_manual_game.py	Test	57
		Total = 2029

3. **Production Code vs User stories/Acceptance Criteria**

User Story ID	User Story Name
1	Choose a board size and type
2	Choose the game mode of a chosen board size and game mode
3	Start a new game of the chosen board size and game mode
4	Make a move in a manual game
5	A manual game is over
6	Make a move in an automated game
7	An automated game is over
8	Randomize the state of the board during a manual game

User Story 1 – Choose a board size and type

User Story ID	AC ID	Acceptance Criterion	Class Name(s)	Method Name(s)	Status
1	1.1	Select a valid board size	SetupScreen	create_widgets(), create_choice_card()	Complete
1	1.2	Change the selected board size	SetupScreen	create_choice_card()	Complete
1	1.3	Select a board type	SetupScreen	create_widgets(), create_choice_card()	Complete
1	1.4	Change the selected board type	SetupScreen	create_choice_card()	Complete

User Story 2 – Choose the game mode of a chosen board

User Story ID	AC ID	Acceptance Criterion	Class Name(s)	Method Name(s)	Status
2	2.1	Select Manual game mode	SetupScreen	create_widgets(), create_choice_card()	Complete
2	2.2	Select Automated game mode	SetupScreen	create_widgets(), create_choice_card()	Complete
2	2.3	Change the selected game mode	SetupScreen	create_choice_card()	Complete

User Story 3 – Start a new game of the chosen board size and game mode

User Story ID	AC ID	Acceptance Criterion	Class Name(s)	Method Name(s)	Status
3	3.1	Start with selected configuration and game mode	SetupScreen, SolitaireGUI, Game	start_game(), __init__()	Complete
3	3.2	Initialize the board	Board, Game, GameScreen, BoardGraphics	create_board(), __init__(), draw_board()	Complete

3	3.3	Start another new game	GameScreen, SolitaireGUI, SetupScreen	main_menu(), return_to_setup(), show_setup_screen(), start_game()	Complete

User Story 4 – Make a move in a manual game

User Story ID	AC ID	Acceptance Criterion	Class Name(s)	Method Name(s)	Status
4	4.1	Execute a valid move	GameScreen, Game	handle_click(), make_moves()	Complete
4	4.2	Reject an invalid move	Game	is_valid_move(), make_moves()	Complete
4	4.3	Execute orthogonal or diagonal jump	Game	get_move_directions(), is_valid_move(), make_moves()	Complete

User Story 5 – A manual game is over

User Story ID	AC ID	Acceptance Criterion	Class Name(s)	Method Name(s)	Status
5	5.1	End with one peg remaining	Game, GameScreen	is_game_won(), check_game_status()	Complete
5	5.2	End when no valid moves remain	Game, GameScreen	is_game_over(), check_game_status()	Complete
5	5.3	Display game rating	GameScreen	check_game_status()	Complete
5	5.4	Display and dismiss the Game Over popup	GameScreen	check_game_status()	Complete
5	5.5	Player chooses to return to the main menu	GameScreen, SolitaireGUI	main_menu(), return_to_setup(), show_setup_screen()	Complete
5	5.6	Player chooses to	GameScreen	exit_game()	Complete

		exit the application			
5	5.7	Player saves the completed game recording	GameScreen, GameRecording	check_game_status(), save_game()	Complete

User Story 6 – Make a move in an automated game

User Story ID	AC ID	Acceptance Criterion	Class Name(s)	Method Name(s)	Status
6	6.1	Automatically identify valid moves	Game, AutomatedGame	available_moves(), is_valid_move(), autoplay_step()	Complete
6	6.2	Automatically select and execute a valid move	AutomatedGame, Game	autoplay_step(), make_moves()	Complete
6	6.3	Update the board after each automated move	GameScreen, BoardGraphics	autoplay_step(), draw_board(), update_peg_count()	Complete
6	6.4	Continue making moves without player input	GameScreen	start_autoplay(), autoplay_step()	Complete
6	6.5	Prevent the player from manually moving pegs	GameScreen	handle_click()	Complete

User Story 7 – An automated game is over

User Story ID	AC ID	Acceptance Criterion	Class Name(s)	Method Name(s)	Status
7	7.1	End with one peg remaining	Game, GameScreen	is_game_won(), check_game_status()	Complete

7	7.2	Stop automated gameplay when no valid moves remain	Game, AutomatedGame, GameScreen	is_game_over(), autoplay_step(), stop_autoplay()	Complete
7	7.3	Display game rating	GameScreen	check_game_status()	Complete
7	7.4	Display and dismiss the Game Over popup	GameScreen	check_game_status()	Complete
7	7.5	Player chooses to return to the main menu	GameScreen, SolitaireGUI	main_menu(), return_to_setup(), show_setup_screen()	Complete
7	7.6	Player chooses to exit the application	GameScreen	exit_game()	Complete
7	7.7	Player saves the completed game recording	GameScreen, GameRecording	check_game_status(), save_game()	Complete

User Story 8 – Randomize the state of the board during a manual game

User Story ID	AC ID	Acceptance Criterion	Class Name(s)	Method Name(s)	Status
8	8.1	Randomly select a board type and size	SetupScreen	randomize_settings()	Complete
8	8.2	Randomly select the starting empty hole	Board, Game	randomize_empty_hole(), __init__()	Complete
8	8.3	Enable or disable board randomization	SetupScreen	randomize_settings(), randomize_appearance()	Complete

8	8.4	Start a game using the randomized configuration	SetupScreen, SolitaireGUI, Game	start_game(), __init__()	Complete
8	8.5	Cancel randomization when manually selecting board settings	SetupScreen	create_choice_card(), randomize_appearance()	Complete

4. Test vs User stories/Acceptance Criteria

4.1 Automated tests directly corresponding to some acceptance criteria

User Story ID	Acceptance Criterion ID	Class Name(s) of Test Code	Method Name(s) of Test Code	Status
1	1.1	TestBoard	test_english_board_size()	Pass
1	1.1	TestBoard	test_diamond_board_size()	Pass
1	1.1	TestBoard	test_hexagon_board_size()	Pass
1	1.3	TestBoard	test_english_board_shape()	Pass
1	1.3	TestBoard	test_diamond_board_shape()	Pass
1	1.3	TestBoard	test_hexagon_board_shape()	Pass
1	1.4	TestGame	test_new_game_changes_board_type()	Pass
2	2.1	TestManualGame	test_manual_game_creation()	Pass
2	2.2	TestAutomatedGame	test_automated_game_creation()	Pass
3	3.1	TestGame	test_game_initialization()	Pass
3	3.2	TestBoard	test_empty_center()	Pass
3	3.3	TestGame	test_new_game_resets_board()	Pass
4	4.1	TestGame	test_valid_move()	Pass
4	4.1	TestGame	test_make_move()	Pass
4	4.1	TestManualGame	test_valid_manual_move()	Pass
4	4.2	TestGame	test_invalid_start_position()	Pass
4	4.2	TestGame	test_invalid_destination()	Pass
4	4.2	TestGame	test_invalid_move_distance()	Pass
4	4.2	TestGame	test_invalid_does_not_change_board()	Pass
4	4.2	TestManualGame	test_invalid_manual_move()	Pass
4	4.3	TestGame	test_make_move(), test_hexagon_diagonal_move()	Pass
5	5.1	TestGame	test_game_won(), test_game_not_won()	Pass
5	5.2	TestGame	test_game_over()	Pass

5	5.1, 5.2	TestManualGame	test_manual_game_over()	Pass
6	6.1, 6.2	TestAutomatedGame	test_autoplay_makes_valid_move()	Pass
6	6.2	TestAutomatedGame	test_autoplay_reduces_peg_count()	Pass
7	7.2	TestAutomatedGame	test_autoplay_stops_when_no_moves_remain()	Pass

4.2 Manual test directly corresponding to some acceptance criteria

User Stories 1-3: Game configuration and initialization

User Story ID	Acceptance Criterion ID	Test Case Input	Test Oracle (Expected Output)	Notes
1	1.1	Select 5×5, 7×7, and 9×9.	Each valid board size can be selected and highlighted.	Setup screen
1	1.2	Change the board size from 7×7 to 9×9.	The 9×9 card becomes highlighted and the 7×7 card is deselected.	Setup screen
1	1.3	Select English, Diamond, and Hexagon.	Each board type can be selected and highlighted.	Setup screen
1	1.4	Change the board type from English to Diamond.	Diamond becomes selected and English is deselected.	Setup screen
1	1.1, 1.3	Execute manual_board_test.py.	All nine combinations of board sizes and types are displayed correctly.	Terminal Test
2	2.1	Select Manual Play.	Manual Play becomes highlighted.	Setup screen

2	2.2	Select Automated Play.	Automated Play becomes highlighted.	Setup screen
2	2.3	Change from Manual to Automated Play.	Automated Play becomes selected and Manual Play is deselected.	Setup screen
3	3.1	Select Manual, Diamond, 9×9, and START GAME.	A manual game begins with a 9×9 Diamond board.	Manual mode
3	3.1	Select Automated, English, 7×7, and START GAME.	An automated game begins with a 7×7 English board.	Automated mode
3	3.2	Start a game without randomization.	The selected board is populated with pegs except for the center hole.	Board initialization
3	3.3	Select MAIN MENU, confirm, and start another game.	A new game begins using the newly selected configuration.	Navigation

User Story 4: Make a move in a manual game

User Story ID	Acceptance Criterion ID	Test Case Input	Test Oracle (Expected Output)	Notes
4	4.1	Select a peg and a valid destination.	The selected peg moves, the jumped peg is removed, and the peg count decreases by one.	Valid move

4	4.2	Select a peg and attempt an illegal jump.	The move is rejected, the board remains unchanged, and an invalid move message appears.	Invalid move
4	4.3	Perform a valid horizontal or vertical jump.	The orthogonal jump is executed successfully.	Orthogonal move
4	4.3	Perform a valid diagonal jump.	The diagonal jump is executed successfully.	Diagonal move

User Story 5: A manual game is over

User Story ID	Acceptance Criterion ID	Test Case Input	Test Oracle (Expected Output)	Notes
5	5.1	Complete a manual game with one peg remaining.	A congratulations message and Outstanding rating are displayed.	Winning condition
5	5.2	Complete a manual game with no valid moves remaining.	The game ends and further moves are prevented.	Game-over condition
5	5.3	Complete a manual game.	The Game Over popup displays the remaining pegs and corresponding rating.	Rating
5	5.4	Click OK on the Game Over popup.	The popup closes and the completed board remains visible.	Popup

5	5.5	Select MAIN MENU and confirm Yes.	The application returns to the setup screen.	Navigation
5	5.6	Select EXIT and confirm Yes.	The application closes.	Exit
5	5.7	Attempt to save an unfinished manual game.	SAVE GAME is disabled.	Recording restriction
5	5.7	Complete a manual game and select SAVE GAME.	A file dialog opens and the completed recording can be saved as a .txt file.	Recording

User Story 6: Make a move in an automated game

User Story ID	Acceptance Criterion ID	Test Case Input	Test Oracle (Expected Output)	Notes
6	6.1	Start an automated game.	The application identifies available valid moves.	Automated mode
6	6.2	Observe the computer performing a move.	A valid move is executed and one peg is removed.	Automated move
6	6.3	Observe the board after an automated move.	The graphical board and remaining peg count are updated.	Graphics
6	6.4	Start an automated game without providing further input.	The computer continues making moves automatically at approximately 500-millisecond intervals.	Autoplay

6	6.5	Click pegs while automated gameplay is running.	Manual input does not select or move pegs.	Input restriction

User Story 7: An automated game is over

User Story ID	Acceptance Criterion ID	Test Case Input	Test Oracle (Expected Output)	Notes
7	7.1	Allow an automated game to finish with one peg remaining.	A congratulations message and Outstanding rating are displayed.	Winning condition
7	7.2	Allow an automated game to run until no valid moves remain.	Automated gameplay stops.	Game-over condition
7	7.3	Allow an automated game to finish.	The remaining peg count and corresponding rating are displayed.	Rating
7	7.4	Click OK on the Game Over popup.	The popup closes and the completed board remains visible.	Popup
7	7.5	Select MAIN MENU and confirm Yes.	The application returns to the setup screen.	Navigation
7	7.6	Select EXIT and confirm Yes.	The application closes.	Exit
7	7.7	Attempt to save while automated gameplay is running.	SAVE GAME is disabled.	Recording restriction

7	7.7	Complete an automated game and select SAVE GAME.	The completed game recording can be saved as a .txt file.	Recording

User Story 8: Randomize the state of the board during a manual game

User Story ID	Acceptance Criterion ID	Test Case Input	Test Oracle (Expected Output)	Notes
8	8.1	Click RANDOMIZE while it is deselected.	A different combination of board type and size is selected.	Randomization
8	8.2	Start several games with RANDOMIZE enabled.	Each board contains exactly one empty starting hole, whose position can vary between games.	Random starting hole
8	8.3	Click RANDOMIZE once.	The button turns blue and randomization is enabled.	Enable
8	8.3	Click RANDOMIZE again.	The button returns to dark gray and randomization is disabled.	Disable
8	8.4	Select Manual Play, enable RANDOMIZE, and select START GAME.	A manual game begins using the randomized configuration and starting hole.	Randomized game
8	8.5	Enable RANDOMIZE, then manually select a board type or size.	Randomization is disabled and the button returns to dark gray.	Cancel randomization

8	8.5	Start a game after canceling randomization.	The selected board starts with an empty center hole.	Default starting hole

4.3 Other automated or manual test not corresponding to the acceptance criteria

Number	Test Input	Expected Result	Class Name of Test Code	Method Name of Test Code
1	Check valid board positions.	Valid positions return True.	TestBoard	test_valid_positions()
2	Check invalid and out-of-bounds board positions.	Invalid positions return False.	TestBoard	test_invalid_positions()
3	Create a board with an unsupported board type.	A ValueError exception is raised.	TestBoard	test_invalid_board_type()
4	Select REPLAY GAME and open a valid recording.	The original board configuration is restored and replay begins.	N/A — Manual	N/A
5	Replay a completed game recording.	The recorded moves execute in their original order, producing the same final board and peg count.	N/A — Manual	N/A
6	Attempt to save during replay.	SAVE GAME remains disabled.	N/A — Manual	N/A
7	Select an invalid recording file.	A Replay Error popup is displayed.	N/A — Manual	N/A
8	Open the recording file in a text editor.	The file contains the board type, board size,	N/A — Manual	N/A

		initial board configuration, and successful moves.		
9	Select MAIN MENU during automated gameplay and choose No.	Automated gameplay pauses during confirmation and resumes afterward.	N/A — Manual	N/A
10	Select EXIT during automated gameplay and choose No.	The application remains open and automated gameplay resumes.	N/A — Manual	N/A
11	Select MAIN MENU during replay and choose No.	Replay pauses during confirmation and resumes afterward.	N/A — Manual	N/A

5. **Part 1: Describe how the class hierarchy in your design deals with the common and different requirements of the Manual Game and the Automated Game.**

The class hierarchy in my Peg Solitaire application was designed to accommodate the common and different requirements of manual and automated gameplay while minimizing code duplication. The Game class serves as the parent class for the ManualGame and AutomatedGame subclasses. Both subclasses inherit the common functionality required to play Peg Solitaire, while the AutomatedGame class contains additional functionality specific to automated gameplay.

The Game class is responsible for implementing the rules and functionality shared between both game modes. This includes initializing the board, validating moves, executing moves, identifying available moves, determining when the game is over, and determining whether the game has been won. For example, the is_valid_move() method determines whether a requested move follows the rules of Peg Solitaire, while the make_moves() method executes a valid move and updates the board. By placing these methods in the parent class, both manual and automated games follow the same rules without requiring separate implementations.

The ManualGame class inherits all of its functionality from the Game class and does not require additional methods. The main difference between manual and automated gameplay is how a move is selected. In manual mode, the player selects a peg and its destination using the graphical interface. The handle_click() method in the GameScreen class processes the player's input and calls the inherited make_moves() method to execute the move.

The AutomatedGame class extends the functionality of the Game class by implementing the autoplay_step() method. This method calls the inherited available_moves() method to identify all possible moves, selects the first available move, and executes it using make_moves(). The GameScreen class schedules these automated moves at regular intervals until no valid moves remain. Unlike manual gameplay, automated gameplay does not require player input to select or execute moves.

Additional functionality, such as recording successful moves and randomizing the starting empty hole, is also implemented through shared functionality in the parent class. This allows both game modes to support these features without duplicating code.

Overall, the class hierarchy improves the maintainability and organization of the application by separating common game functionality from mode-specific behavior. If the rules of Peg Solitaire need to be modified, the changes can be made in the Game class rather than separately in both subclasses. This design also allows additional game modes to be introduced in the future while reusing the existing game logic.

6. **Part 2: Demonstrate how you use LLM (ChatGPT or other) to analyze how well your code adheres to the design principles discussed in class – modularity, cohesion, coupling, and encapsulation – using the definitions provided in class. Provide screenshots of your interactions with the LLM, showing the Interpret the LLM's feedback, refine your prompt if needed to obtain more relevant responses, and discuss any changes made to your code based on the analysis. Explain how these refinements improve adherence to the design principles.**

   Using ChatGPT, I prompted it to summarize how well my code adheres to the design principles: modularity, cohesion, coupling, and encapsulation. In response to my prompt, ChatGPT provided its assessment along with examples from my code to support its conclusions regarding adherence to each design principle.

   Based on the feedback provided by ChatGPT, my code demonstrates strong adherence to modularity and cohesion, moderate to strong adherence to coupling, and moderate adherence to encapsulation. I agree with the assessment of modularity because I separated the different responsibilities of my application into individual classes and files. This also contributes to high cohesion because most of my classes have a specific purpose.

   One improvement suggested by ChatGPT was to separate some of the responsibilities of the GameScreen class. Currently, this class is responsible for several different features, including manual gameplay, automated gameplay, replay, saving recordings, and navigation. ChatGPT recommended separating replay and automated playback scheduling into dedicated components. I agree that implementing this recommendation would improve modularity and cohesion by reducing the number of responsibilities within the GameScreen class and allowing each component to focus on a specific purpose. However, I also agree that this could introduce unnecessary complexity considering the size of the project.

   After receiving the initial detailed analysis, I refined my prompt by asking ChatGPT to summarize its assessment of each design principle. This provided a more concise response that made it easier to identify the strengths and weaknesses of my current design.

   Another design decision I evaluated with ChatGPT was whether the ManualGame subclass was necessary since it does not contain any additional methods or attributes. Although removing this class would simplify the class hierarchy, I decided to retain it because it provides a clear distinction between manual and automated gameplay and allows for future expansion of manual-specific functionality.

After evaluating the feedback, I decided not to make additional changes to my code at this time. Although separating the responsibilities of the GameScreen class could further improve modularity and cohesion, I believe the current design is appropriate for the size and requirements of the project. If the application were to expand and introduce additional functionality in the future, I would consider implementing these recommendations to improve maintainability and reduce complexity within individual classes.



